Government College of Engineering, Jalgaon

(An Autonomous Institute of Govt. of Maharashtra)

Department of Computer Engineering

Student Name:_____PRN_____

Batch:_____Date of Performance_____ Date **of Completion_____**

Experiment No. 6

Title: To write a program to implement Wine Classification using Back propagation.

Theory

Wine tasting is a fine art which enables classification of Wine. When it comes to classification of wine, the practice is quite varied based on region of origin and time. It is one of the most tasteful traditions which is also protected by law of its own in certain regions. The classification varies based on vintage, sweetness, appellation , vinification styles, varietal or blend.

Is it possible to teach something about the classification of different variety of wines to Neural Networks? Well, I intend to do exactly that and get hands on with code as well.

In order to train a Neural Network on how to classify wine, we need to provide it a set of data labels that describes a variety of attributes that has real values for the attributes of wine. Since the training shall be supervised, we need to ensure the wine "classes" are also part of the training data so that we "teach" the Neural Nets about the attributes.

The Wine Dataset

I shall use the UC Irvine Machine Learning repository, which has a wine dataset having 13 attributes based off a chemical analysis done for different wine in the Italy region. The wines are from 3 different cultivars and hence has distinct chemical compositions that helps identify which cultivar is the wine from.

The link to the wine data set is > here

The characteristics of the data set is as follows (reproduced from the UCI link)
Data Set Characteristics: Multivariate
Number of Instances: 178

Attribute Characteristics: Integer, Real
Number of Attributes: 13
Missing Values? None

The names of the attributes are as follows:

Alcohol
Malic acid
Ash
Alcalinity of ash
Magnesium
Total phenols
Flavanoids
Nonflavanoid phenols
Proanthocyanins
Color intensity
Hue

OD280/OD315 of diluted wines
Proline
The data values, which are in a comma separated format for these attributes are available in the UCI data folder > here

A sample of the data shall look like this:
1,14.23,1.71,2.43,15.6,127,2.8,3.06,.28,2.29,5.64,1.04,3.92,1065
2,12.37,.94,1.36,10.6,88,1.98,.57,.28,.42,1.95,1.05,1.82,520

3,12.25,3.88,2.2,18.5,112,1.38,.78,.29,1.14,8.21,.65,2,855

...

Note that the first value describes the "class" of the data. In other words, it is the ID of the cultivar. For the sake of this exercise, we don't need the exact name or locality of the cultivar. We are fine with the data, as long as the value for the next 13 attributes are made available.

Now, lets design the Neural Network, that shall taste the wine and classifiy the wine to the correct cultivar.

Neural Network Type and Size

We need to determine the type of network, the number of input, hidden and output neurons and type of activation functions to use.

We shall use a standard Multilayer Feedforward Neural Network since this is good enough for classification tasks.

From the dataset, we know that there are 13 attributes, so we shall use 13 input neurons for the network.

Also, the total number of cultivars are 3, so we shall use 3 output neurons for the network to point to. The output shall be mutually exclusive (Only one output neuron shall be activated, denoting the correct cultivar for the input value)

Why are we using 3 separate output neurons as against using 1 output neuron to learn the 3 different IDs? Well, because the IDs themselves don't have any real meaning. The value 1, 2, 3 can be replaced

with 22, 23, 24 to represent the cultivar anytime and the dataset still holds. This is the characteristic of a "Multivariate" dataset.

Now, how many hidden neurons are needed? Let's use our learnings from the previous post "is Optimizing your Neural Network a Dark-Art?" which provides us a thumb rule as follows:

$$N_h = \frac{N_s}{\propto * (N_i + N_o)}$$

As per the stated thumb rule we get:

$$\frac{178}{2*(13+3)} = 5.56$$

The equation is self explanatory. I am using an 'alpha' of 2 since the inputs samples are low. Lets round up the hidden neurons needed as 6.
Feature Scaling : Or, Normalizing the Inputs
Before we design the activities of the network, its prudent to learn the characteristics of the input data. Without really understanding the data, there is no way you can design the activities of the Neural Net.
Note: All the values for input and output in a Neural Network MUST always be numerical.
Some of the characteristics of the input data can be:
The features maybe integers, or a combination of real and integers. (discrete versus continuous)
Different feature may vary in size. Some features can be in low fractions while others can be very large numbers. (across features)
Each feature by themselves may have varied values ranging from low to high. (within the feature)
Features can be sparse (as in only some may contain values)
Features can be dense (all of it may contain values)
Given this, its important to pre-process the input data. In the wine dataset, notice that you have real values ranging from 0.1 for some features and above 1000 for others. While all of this data is not correlated in its own, it is important in machine-learning to scale down the data to a meaningful number.
Specifically for Neural Network, the transfer potential is a inner-product of the input data to the weights. Since all input neurons is connected to every single hidden neuron, the number of fan-ins, and the value of the input neuron can get very large quickly. Even if you are not using a dot-product for your transfer potential function but decide to use some Radial Basis Function, the distance measure shall be very large (Euclidean distance) for the activation functions.
Also, if one of the features land up having a very broad range of value (Lets say a particular attribute has a value ranging from 10 to 10,000), then the rest of the distance of other variables will be governed by the particular feature (either Euclidian in case of a RBF or the vector length in case of a dot-product).

Its important to scale down the range of all the features so that each feature contributes proportionally to the final distance (to the vector length or some other metric)
Also, note that in Neural Networks, since we use Gradient Descent to reduce the empirical error, scaling down the range helps the network converge faster.

So, what are some of the techniques for Feature Scaling?
Rescaling the feature : Rescaling the feature is a technique that sets the feature to a range between {-1, 1} based on the maximum value and the minimum value of the range within the feature. The Rescaling function is as follows:

$$x' = \frac{x - \bar{x}}{x_{max} - x_{min}}$$

Rescaling Function
Here x-prime is the rescaled value
x-bar is the average, x-max is the maximum and x-min is the minimum in the range of possible values.
Zero-mean, unit-variance : You can also standardize your feature using a standard-score by determining the mean of the distribution within the range of possible values and the standard deviation. This is typically the most popular method used in other machine learning techniques such as Logistic-Regression or Support Vector Machines where margins for decision-boundaries has to be calculated. The equation for Zero-mean, unit variance is as follows:

Zero-mean, unit variance

$$x' = \frac{x - \bar{x}}{\sigma}$$

- The subtraction of the mean from the original value (Similar to Rescaling) makes it Zero-mean.
- The division by standard deviation makes it unit variance.

Unit Length : Another technique is to reduce the feature range to a unit length (as against a unit-variance). Here, we normalize the length of the feature-range-vector to unit length 1. In other words, the range of features add up to 1. To do so, we divide each component value within the feature-range-vector by its Euclidean distance. The equation is as follows:

Unit-length function

- The denominator represents the Euclidean distance, which is the root of sum of squares of all component values in the range of the feature.

$$x' = \frac{x}{\sqrt[2]{\sum_{i=1}^{n} x_i^2}}$$

How should we normalize the wine.data?
Here is a snapshot of the maximum and minimum values of the range in our features :

	Max	Min
Alcohol	14.83	11.03
Malic acid	5.8	0.74
Ash	3.23	1.36
Alcalinity of ash	30	10.6
Magnesium	162	70
Total phenols	3.88	0.98
Flavanoids	5.08	0.34
Nonflavanoid phenols	0.66	0.13
Proanthocyanins	3.58	0.41
Color intensity	13	1.28
Hue	1.71	0.48
OD280/OD315 of diluted wines	4	1.27
Proline	1680	278

In the raw data, we can notice that the Proline feature has large values above 1000, while we have

other features which are in fractions (Hue, Malic Acid, Total Phenols). This is quite a variance. Also the range (variance) within Proline is ranging from 278 to 1680. This seems true for Magnesium and Color intensity as well.
Given this, its prudent that we choose Zero-mean, unit-variance to normalize the raw data to given us a standard-score to work with the data.
Design of Network Activities
Now that we have got the type and size of network, we need to determine the activation functions for the network. We looked at activation functions in the previous post titled "Mathematical Foundation for Activation Functions"
From the data, we understand that the data is multivariate classification with 3 classes. Also there are about 178 samples (this is not huge) and 13 input features (not big again).
Design of the Output Activities: For multivariate classification, we can choose the output layer activation function as softmax activation.
Why Softmax? : The output classifiers are multivariate and mutually exclusive in our example. In other words, the wine shall be classified into one and ONLY one class without a overlap.
We would need a mechanism to choose the right class with the highest degree of probability.
In other words, given a set of 13 input values, we need to determine the possibility (or probability) of the inputs to belong to either class 1, class 2, or class 3.
In other words, we are stating that the output is mutually exclusive and collectively exhaustive. Hence :

$$P(A \text{ or } B \text{ or } C) = P(A) + P(B) + P(C)$$

Here A or B or C represents the cultivars.
The softmax function is a quashing function, which takes a k-dimensional vector of any arbitrary real-values and converts that to a k-dimensional vector of real values in the range {0,1} which sums totally to 1.
The softmax function is given as follows:

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^{k} e^{z_k}} \; for\; j = 1, \ldots, k$$

Softmax function.
The softmax equation can be used in any probabilistic multiclass classification including multinomial regression.
Design of the Hidden Activities
The input activity has a choice of being a sigmoid, hyperbolic tan or a ReLU.
The sigmoid or the hyperbolic tanh has the tendency to quash the output of the hidden activity to a range between {0,1} or {-1,1} respectively. While this is useful, given that we are using a zero-mean-unit variance as a normalizer for the inputs and also given that we are using a softmax (as explained) for the output activity, it does not makes sense to restrict the hidden activity to a range. Instead, we may want to amplify the feature-differences in the input feature vector when its above a zero threshold so that the softmax gets to work with a linear scale when the input value is above zero.

A nonlinear function, which becomes linear post zero-threshold is your Rectified Linear units. The function of the ReLU is quite easy to differentiate and the activity by itself is quite light as follows:

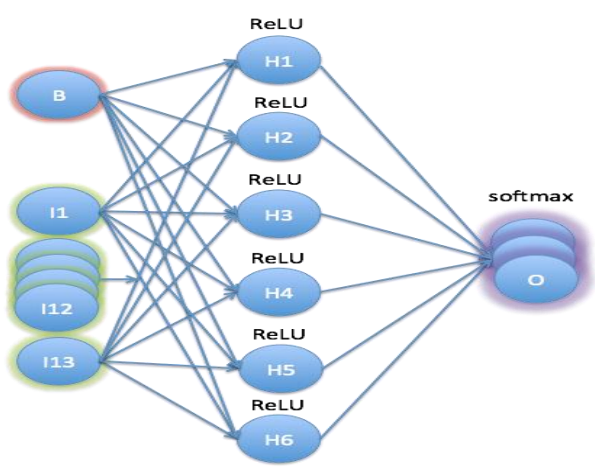
$$f(x) = \begin{cases} 0 & \text{for} \quad x < 0 \\ x & \text{for} \quad x \geq 0 \end{cases}$$

Rectified Linear Unit
Hence, we shall choose ReLU for the hidden activities.
Network Optimization Consideration
We shall keep the Stochastic Gradient Descent (standard) for the empirical error reduction as explained >
We shall also use the Negative Log Likelihood as our Cost function as explained >
We shall use a momentum (Also called the Nesterov Momentum) for dampening the velocity of learning as explained (momentum value; "alpha") >
So the backpropagation equation along with a learning rate and the momentum value shall look like this:

$$\Delta \boldsymbol{w}_{(t)ij} = \epsilon \frac{\partial E}{\partial w_{ij}} + \propto \Delta \boldsymbol{w}_{(t-1)ij}$$

$$\boldsymbol{w}_{ij} = \boldsymbol{w}_{ij} + \Delta \boldsymbol{w}_{(t)ij}$$

Backprop with learning rate and momentum-value
Code Showdown
Given that we have considered the pre-processing, network type, size and optimization procedures, its time to get to syntax.
Note: I am using DL4J as my Neural Network framework (I have explained why I am doing so >)
The full code for the classifier can be found here > [WineClassfier.java]
The code sets up a network as follows:



Implementation:
There are 13 input neurons ranging from {I1….I13}, 6 hidden neurons {H1…H6}, and 3 output neurons {O1.. O3}. While, this is not the best picture to show the entirety of the architecture, you get the idea, 65% of the file to train the Neural Network, while the remaining as test data for validation.
Stochastic Gradient Descent for optimization, a learning rate Epsilon of 5% on the error derivative and a momentum value alpha of 10% on the previous error delta to be used by the backpropagation algorithm.
The hidden neuron gets a ReLU activation and the output neurons get a softmax activation. The cost function is a negative log likelihood.

Classified Wine Data on the Test Set
The final output actually uses the 35% of the wine.data file that was set aside to test the Neural Network. Notice that there were about 13 class zero data, 26 class 1 data and 24 class 2 data that was used.
Good news is that every single test data that was used has been converged to a positive result.
It states, “examples labeled as 1 classified by model as 1: 26 times” which means that what should be classified as class-1 input is actually classified correctly as class-1.

Code :

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Load the wine dataset
data = load_wine()

# Split data into features and target
X = data.data
y = data.target

# Preprocess the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Encode the target labels
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_encoded, test_size=0.2,
random_state=42)
```

```
# Define the neural network model
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(64, activation='relu'),
    Dense(3, activation='softmax')  # 3 output neurons for 3 wine classes
])

# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(X_train, y_train, epochs=100, batch_size=32, validation_split=0.2)

# Evaluate the model on the test set
loss, accuracy = model.evaluate(X_test, y_test)
print(f'Test Accuracy: {accuracy}')
```

Output :

```
PS F:\Bhargav_Data\Documents\8th_Sem\FANN\FANNL> python .\practical_5.py
Epoch 1/50
4/4 [==============================] - 2s 123ms/step - loss: 5.6739 - accuracy: 0.5000 - val_loss:
2.3836 - val_accuracy: 0.4688
Epoch 2/50
4/4 [==============================] - 0s 26ms/step - loss: 1.4643 - accuracy: 0.4766 - val_loss:
1.8016 - val_accuracy: 0.5312Epoch 3/50
4/4 [==============================] - 0s 21ms/step - loss: 2.1939 - accuracy: 0.5000 - val_loss:
1.8768 - val_accuracy: 0.5312Epoch 4/50
                                                    …….
0.0186 - val_accuracy: 1.0000Epoch 98/100
4/4 [==============================] - 0s 21ms/step - loss: 0.0022 - accuracy: 1.0000 - val_loss:
0.0186 - val_accuracy: 1.0000Epoch 99/100
4/4 [==============================] - 0s 19ms/step - loss: 0.0021 - accuracy: 1.0000 - val_loss:
0.0186 - val_accuracy: 1.0000Epoch 100/100
4/4 [==============================] - 0s 21ms/step - loss: 0.0021 - accuracy: 1.0000 - val_loss:
0.0185 - val_accuracy: 1.00002/2 [==============================] - 0s 6ms/step - loss: 0.0068 -
accuracy: 1.0000
Test Accuracy: 1.0
PS F:\Bhargav_Data\Documents\8th_Sem\FANN\FANNL>
```

Conclusion:- In this practical we implemented wine classification using back propagation algorithm

Ms. Diksha R. Gupta
Course Teacher